

TRANSACTION MANAGEMENT

Learning Objectives:

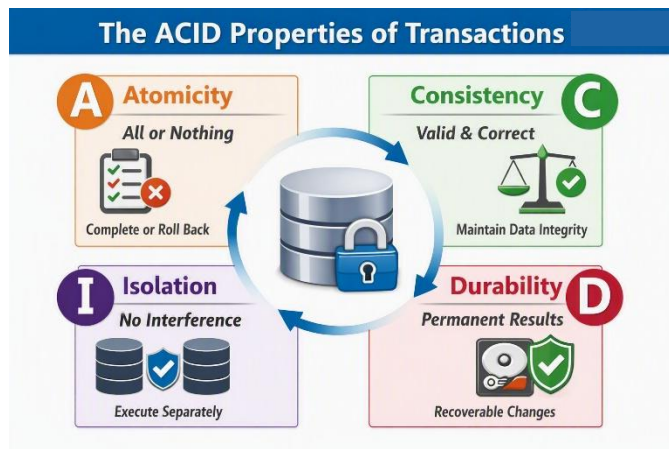
- Define what a **transaction** is in database systems.
- Explain the four key properties of transactions: **Atomicity, Consistency, Isolation, Durability (ACID)**.
- Understand why transactions are essential for maintaining database reliability and integrity.
- Distinguish between **volatile, non-volatile, and stable storage**.
- Explain how storage types affect **atomicity** and **durability** of transactions.
- Understand why stable storage is critical for reliable database systems.

Transactions

- A **transaction** is a unit of program execution that accesses or updates data items.
- Typically initiated by user programs written in SQL, C++, or Java with embedded database calls.
- Transactions are **delimited** by BEGIN TRANSACTION and END TRANSACTION.
- All operations between these boundaries form a single, indivisible unit.

The ACID Properties

Transactions must satisfy four fundamental properties:



1. **Atomicity (All-or-None Execution)**
 - A transaction must execute completely or not at all.
 - If a failure occurs (system crash, error, etc.), partial changes must be undone. It must go back to the exact way it was before the task started. This rule keeps your data safe and correct.
 - Ensures the database is never left in an inconsistent state.
 - **Example:** If a bank transfer fails midway, no money is deducted.
2. **Consistency (Preserving Database Rules)**
 - A transaction must transform the database from one consistent state to another.
 - Goes beyond basic integrity constraints (primary keys, referential integrity).
 - Responsibility lies with the programmer to enforce application-specific rules.
 - **Example:** Ensures account balances remain accurate after updates.
3. **Isolation (No Interference)**
 - Transactions should appear to execute independently, without being affected by other concurrent operations.
 - Even though multiple transactions may run simultaneously, the system ensures they don't interfere.
 - **Example:** Two users withdrawing funds simultaneously don't conflict.

4. Durability (Persistence of Results)

- Once a transaction is committed, its effects must survive system crashes.
- Ensures that completed work is not lost.
- **Example:** Once a payment is confirmed, it remains recorded permanently.

Why Transactions Matter

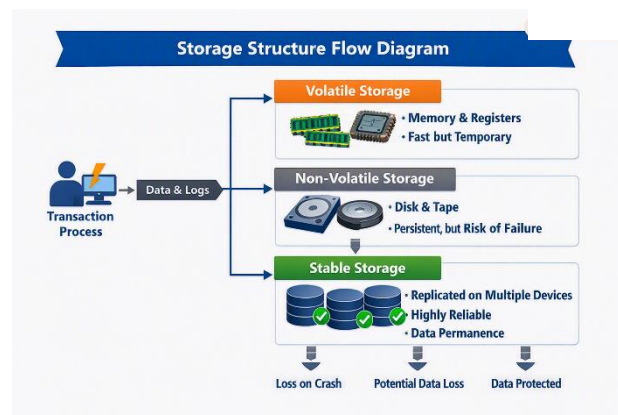
- Transactions provide a **safe and reliable framework** for interacting with databases.
- They protect against errors, crashes, and concurrency issues.
- By enforcing ACID properties, transactions guarantee **data integrity and trustworthiness**.

Key Takeaways

- Transactions are the **building blocks of reliable database systems**.
- ACID properties ensure that operations are **atomic, consistent, isolated, and durable**.
- Proper transaction design is critical for applications that depend on accurate and persistent data.

Storage Structures in Database Systems

- To ensure **atomicity** and **durability**, we must understand how data is stored and accessed.
- Storage media differ in **speed, capacity, and resilience to failure**.
- Classified into three categories: **volatile, non-volatile, and stable storage**.



Types of Storage

1. Volatile Storage

- Data does **not survive system crashes**.
- Examples: **main memory, cache memory**.
- Very fast access, direct retrieval of data items.

2. Non-Volatile Storage

- Data **survives system crashes**.
- Examples: **magnetic disks, flash storage (secondary); optical media, tapes (tertiary)**.
- Slower than volatile storage, especially for random access.
- Susceptible to physical failures.

3. Stable Storage

- Designed so data is **never lost** (practically, extremely unlikely).
- Implemented by **replicating data across multiple non-volatile media** with independent failure modes.
- Updates must be carefully managed to avoid data loss during failures.
- Example: RAID systems with battery backup approximating stable storage.

Role in Transactions

- **Durability:** Transaction changes must be written to **stable storage**.
- **Atomicity:** Log records must be stored in **stable storage before database changes**.

IT ELEC – Advanced Information Management

- Reliability depends on how closely stable storage approximates the ideal.
- High-value applications often require **multiple copies** for stronger guarantees.

Key Takeaways

- **Volatile storage** is fast but temporary.
- **Non-volatile storage** is persistent but vulnerable.
- **Stable storage** is the closest approximation to permanent safety.
- Ensuring atomicity and durability depends on **robust stable storage implementation**.